

动态规划

一维基础：

70 爬楼梯

53 最大子数组和

198 打家劫舍

213 打家劫舍 II

91 解码方法

152 乘积最大子数组

网格：

62 不同路径

63 不同路径 II

64 最小路径和

221 最大正方形

背包：

279 完全平方数

322 零钱兑换

416 分割等和子集

494 目标和

518 零钱兑换 II

字符串：

10 正则表达式匹配

72 编辑距离

139 单词拆分

718 最长重复子数组

1143 最长公共子序列

股票：

121 股票 I

122 股票 II

123 股票 III

188 股票 IV

309 冷冻期

714 手续费

区间：

5 最长回文子串

516 最长回文子序列

647 回文子串

312 戳气球

树形：

337 打家劫舍 III

概念

当前问题能不能由更小的问题推出来，并且这些更小问题会被重复使用。

且满足无后效性（状态转移能构成DAG）

因为大问题可以由小问题推出来，而且这些小问题会重复出现，所以我们把小问题答案保存起来，避免重复计算。

怎么判断一道题是不是 DP？

1. 题目是不是问最值、方案数、可行性、概率？
2. 大问题能不能拆成小问题？
3. 小问题会不会重复出现？
4. 当前状态能不能概括过去，保证后续不用关心具体历史？
5. 是否存在选择，并且选择会影响后续？

最大 / 最小

方案数

是否可达

最长 / 最短

有多少种方法

从前面状态推当前状态

不能贪心直接决定

比如：

最大利润

最少硬币数

不同路径数量

最长公共子序列

最长递增子序列

能否组成目标和

这些大概率是 DP

动态规划做题步骤

1. 定义状态 dp

状态： 可以用数字精确描述的局面，并加上特定的问题场景称之为状态。

2. 找状态转移方程

$dp[i]$ 可以从哪些之前的状态推出来？要构成DAG。

3. 初始化

初始化就是最小问题的答案。要注意边界条件。

4. 遍历顺序

看状态转移依赖谁。

5. 返回结果

题目要的答案在哪个状态里？

动态规划的写法

1. 暴力递归

以爬楼梯为例：

```
int DFS(int n) {  
    if (n == 1) return 1;  
    if (n == 2) return 2;  
  
    return DFS(n - 1) + DFS(n - 2);  
}
```

含义：

爬到第 n 阶的方法数

=

爬到第 $n-1$ 阶的方法数

+

爬到第 $n-2$ 阶的方法数

但它会重复计算。

比如：

DFS(5)

└─ DFS(4)

| └─ DFS(3)

| └─ DFS(2)

└─ DFS(3)

DFS(3) 被重复算了。

所以暴力递归一般会超时。

2. 记忆化搜索

加一个缓存数组：

```
public int ClimbStairs(int n) {  
    int[] memo = new int[n + 1];  
    return DFS(n, memo);  
}  
  
private int DFS(int n, int[] memo) {  
    if (n == 1) return 1;  
    if (n == 2) return 2;  
  
    if (memo[n] != 0) {  
        return memo[n];  
    }  
  
    memo[n] = DFS(n - 1, memo) + DFS(n - 2, memo);  
    return memo[n];  
}
```

这个就是：自顶向下 DP

特点：

从大问题递归到小问题

算过的结果存起来

下次直接用

3. 递推 DP

把递归改成从小到大填表：

```
public int ClimbStairs(int n) {  
    if (n == 1) return 1;  
    if (n == 2) return 2;  
  
    int[] dp = new int[n + 1];  
  
    dp[1] = 1;  
    dp[2] = 2;  
  
    for (int i = 3; i <= n; i++) {  
        dp[i] = dp[i - 1] + dp[i - 2];  
    }  
  
    return dp[n];  
}
```

这个就是：自底向上 DP

DP 最容易错的点

1. 状态定义不清

比如：

dp[i] 是前 i 个？

还是以下标 i 结尾？

还是到 i 为止的最优？

这三种完全不一样。

例如 LIS：

dp[i] 表示以 nums[i] 结尾的最长递增子序列长度

不是：

前 i 个数的最长递增子序列长度

2. 返回值搞错

有些返回：

dp[n]

有些返回：

`max(dp)`

比如最大子数组和、最长递增子序列，通常要返回全局最大值。

3. 初始化错

比如 `dp[0]`、`dp[1]`。

如果初始化不对，后面全错。

4. 遍历顺序错

尤其是背包。

01 背包倒序

完全背包正序

5. 边界条件没处理

比如数组为空、长度为 1、目标为 0。

状态定义常见形式

1. 前 i 个 / 到第 i 个

这是最常见的一维 DP。

形式：

`dp[i]` 表示前 i 个元素的答案

或者：

`dp[i]` 表示到达第 i 个位置的答案

例子：**爬楼梯**

`dp[i]` 表示爬到第 i 阶的方法数

转移：

$$dp[i] = dp[i - 1] + dp[i - 2]$$

打家劫舍

$dp[i]$ 表示考虑前 i 间房子，能偷到的最大金额

2. 以 i 结尾

这种非常重要，尤其是子数组、子序列问题。

形式：

$dp[i]$ 表示以 $nums[i]$ 结尾的答案

注意它和“前 i 个的答案”不一样。

最长递增子序列

$dp[i]$ 表示以 $nums[i]$ 结尾的最长递增子序列长度

转移：

如果 $nums[j] < nums[i]$

$dp[i] = \max(dp[i], dp[j] + 1)$

答案不是 $dp[n - 1]$ ，而是：

$\max(dp[i])$

最大子数组和

$dp[i]$ 表示以 $nums[i]$ 结尾的最大子数组和

转移：

$dp[i] = \max(nums[i], dp[i - 1] + nums[i])$

3. 二维前缀状态

两个字符串、两个数组、二维网格常用。

形式：

$dp[i][j]$ 表示第一个对象前 i 个、第二个对象前 j 个的答案

最长公共子序列

$dp[i][j]$ 表示 $text1$ 前 i 个字符和 $text2$ 前 j 个字符的最长公共子序列长度

如果当前字符相等：

$$dp[i][j] = dp[i - 1][j - 1] + 1$$

否则：

$$dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$$

编辑距离

$dp[i][j]$ 表示 $word1$ 前 i 个字符变成 $word2$ 前 j 个字符的最少操作数

4. 网格位置状态

路径类题常用。

形式：

$dp[i][j]$ 表示到达格子 (i, j) 的答案

不同路径

$dp[i][j]$ 表示走到 (i, j) 的路径数

转移：

$$dp[i][j] = dp[i - 1][j] + dp[i][j - 1]$$

最小路径和

$dp[i][j]$ 表示走到 (i, j) 的最小路径和

转移：

$$dp[i][j] = \min(dp[i - 1][j], dp[i][j - 1]) + grid[i][j]$$

5. 区间状态

看到“子串、区间、从 i 到 j ”要想到区间 DP。

形式：

$dp[i][j]$ 表示区间 $[i, j]$ 的答案

最长回文子序列

$dp[i][j]$ 表示 $s[i..j]$ 中最长回文子序列长度

如果：

$$s[i] == s[j]$$

则：

$$dp[i][j] = dp[i + 1][j - 1] + 2$$

否则：

$$dp[i][j] = \max(dp[i + 1][j], dp[i][j - 1])$$

戳气球

$dp[i][j]$ 表示开区间 (i, j) 内能获得的最大分数

这类题常见套路是：

枚举区间长度

枚举左端点

枚举中间分割点

6. 背包状态

看到“选 / 不选”“容量”“能否组成”“最少数目”“方案数”，要想到背包。

形式：

$dp[i][j]$ 表示前 i 个物品，在容量 j 下的答案

或者压缩成：

$dp[j]$ 表示容量为 j 时的答案

01 背包

每个物品只能选一次。

$dp[j]$ 表示容量为 j 时的最大价值

转移：

```

for (int i = 0; i < n; i++) {
for (int j = bagSize; j >= weight[i]; j--) {
dp[j] = Math.Max(dp[j], dp[j - weight[i]] + value[i]);
}
}

```

容量倒序，因为每个物品只能用一次。

完全背包

每个物品可以无限选。

```

for (int i = 0; i < n; i++) {
for (int j = weight[i]; j <= bagSize; j++) {
dp[j] = Math.Max(dp[j], dp[j - weight[i]] + value[i]);
}
}

```

容量正序，因为允许重复使用当前物品。

7. 状态机 DP

如果题目里有“持有 / 不持有”“买入 / 卖出”“冷冻期”“次数限制”这种状态，就用状态机。

形式：

$dp[i][state]$ 表示第 i 天处于某种状态下的最大收益

股票问题

$dp[i][0]$ 表示第 i 天结束后不持有股票的最大利润

$dp[i][1]$ 表示第 i 天结束后持有股票的最大利润

转移：

$dp[i][0] = \max(dp[i - 1][0], dp[i - 1][1] + prices[i])$

$dp[i][1] = \max(dp[i - 1][1], -prices[i])$

如果有多次交易、冷冻期、手续费，就继续增加状态。

8. 树形 DP

如果数据结构是树，状态一般定义在节点上。

形式：

$dp[node]$ 表示以 $node$ 为根的子树的答案

或者：

$dp[node][0]$ 表示不选 $node$ 的答案

$dp[node][1]$ 表示选 $node$ 的答案

打家劫舍 III

$dp[node][0]$ 表示不偷当前节点时的最大金额

$dp[node][1]$ 表示偷当前节点时的最大金额

转移：

偷当前节点：不能偷左右孩子

不偷当前节点：左右孩子偷不偷都可以

9. 状压 DP

如果题目规模很小，比如 $n \leq 20$ ，并且涉及“选择集合”，可能是状态压缩。

形式：

$dp[mask]$ 表示当前已经选择的集合为 $mask$ 时的答案

例如：

$mask = 10110$

表示某些元素已经被选过。

常见于：

旅行商问题

访问所有节点最短路

分配任务

集合划分

10. 记忆化搜索状态

有些 DP 不好正向填表，可以先写递归。

形式：

dfs(参数1, 参数2, ...)

表示在这些参数确定时的答案

然后用 memo 缓存。

单词拆分：

dfs(start) 表示从 s[start] 开始能否被拆分

如果 dfs(start) 被重复调用，就缓存：

memo[start]

这本质上也是 DP。

空间压缩

如果当前状态只依赖少量历史状态，就没必要保存整个 dp 数组。

当前状态只依赖固定范围内的上一层状态，可以压缩。

如果依赖很多历史状态，或者需要还原路径，就不适合直接压。

1. 一维 DP 压缩成变量

打家劫舍

原始 DP:

```
public int Rob(int[] nums) {  
    int n = nums.Length;  
  
    if (n == 1) return nums[0];  
  
    int[] dp = new int[n];  
  
    dp[0] = nums[0];  
    dp[1] = Math.Max(nums[0], nums[1]);  
  
    for (int i = 2; i < n; i++) {  
        dp[i] = Math.Max(dp[i - 1], dp[i - 2] + nums[i]);  
    }  
  
    return dp[n - 1];  
}
```

状态转移:

$$dp[i] = \max(dp[i - 1], dp[i - 2] + \text{nums}[i])$$

只依赖:

$dp[i - 1]$
 $dp[i - 2]$

所以可以压缩:

```
public int Rob(int[] nums) {  
    int n = nums.Length;  
  
    if (n == 1) return nums[0];  
  
    int prev2 = nums[0];  
    int prev1 = Math.Max(nums[0], nums[1]);  
  
    for (int i = 2; i < n; i++) {  
        int cur = Math.Max(prev1, prev2 + nums[i]);  
  
        prev2 = prev1;  
        prev1 = cur;  
    }  
  
    return prev1;  
}
```

```
    }  
  
    return prev1;  
}
```

2. 二维 DP 压缩成一维

不同路径

二维写法：

```

public int UniquePaths(int m, int n) {
    int[,] dp = new int[m, n];

    for (int i = 0; i < m; i++) {
        dp[i, 0] = 1;
    }

    for (int j = 0; j < n; j++) {
        dp[0, j] = 1;
    }

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i, j] = dp[i - 1, j] + dp[i, j - 1];
        }
    }

    return dp[m - 1, n - 1];
}

```

转移:

$$dp[i][j] = dp[i - 1][j] + dp[i][j - 1]$$

当前格子依赖:

上方

左方

如果压缩成一维:

$dp[j]$ 原来表示上一行的 $dp[i - 1][j]$

$dp[j - 1]$ 表示当前行左边的 $dp[i][j - 1]$

所以:

```

public int UniquePaths(int m, int n) {
    int[] dp = new int[n];

    for (int j = 0; j < n; j++) {
        dp[j] = 1;
    }
}

```



```

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] = dp[j] + dp[j - 1];
        }
    }

    return dp[n - 1];
}

```

这里：

$dp[j] = dp[j] + dp[j - 1];$

含义是：

新的 $dp[j]$

=

旧的 $dp[j]$ ，也就是上方

+

新的 $dp[j - 1]$ ，也就是左方

如果：

$dp[i][j]$ 依赖 $dp[i - 1][j]$ 和 $dp[i][j - 1]$

通常可以从左到右遍历：

```

for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++) {
        dp[j] = dp[j] + dp[j - 1];
    }
}

```

因为：

$dp[j]$ 还没更新前，是上一行

$dp[j - 1]$ 已经更新后，是当前行左边

常见压缩

背包空间压缩

二维定义：

$dp[i][j]$ 表示前 i 个物品，容量为 j 时的最大价值

转移：

不选第 i 个物品： $dp[i - 1][j]$

选第 i 个物品： $dp[i - 1][j - \text{weight}[i]] + \text{value}[i]$

所以：

$dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - \text{weight}[i]] + \text{value}[i])$

压缩成一维：

$dp[j]$ 表示容量为 j 时的最大价值

代码：

```
for (int i = 0; i < n; i++) {  
    for (int j = bagSize; j >= weight[i]; j--) {  
        dp[j] = Math.Max(dp[j], dp[j - weight[i]] + value[i]);  
    }  
}
```

为什么 j 要倒序？

因为每个物品只能用一次。

如果正序：

```
for (int j = weight[i]; j <= bagSize; j++)
```

那么：

$dp[j - \text{weight}[i]]$

可能已经被当前物品更新过了，就会导致同一个物品被重复使用。

所以：

01 背包：容量倒序

完全背包空间压缩

完全背包每个物品可以用无限次。

所以容量正序：

```
for (int i = 0; i < n; i++) {  
    for (int j = weight[i]; j <= bagSize; j++) {  
        dp[j] = Math.Max(dp[j], dp[j - weight[i]] + value[i]);  
    }  
}
```

为什么正序？

因为完全背包允许重复使用当前物品。

正序时：

`dp[j - weight[i]]`

可能已经包含当前物品，正好表示可以继续选当前物品。

所以：完全背包：容量正序

字符串二维 DP 压缩-最长公共子序列

$dp[i][j]$ 表示 text1 前 i 个和 text2 前 j 个的 LCS 长度

转移:

如果相等:

$$dp[i][j] = dp[i - 1][j - 1] + 1$$

否则:

$$dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$$

这里依赖:

上方 $dp[i - 1][j]$

左方 $dp[i][j - 1]$

左上 $dp[i - 1][j - 1]$

压缩时要额外保存左上角。

```
public int LongestCommonSubsequence(string text1, string text2) {
    int m = text1.Length;
    int n = text2.Length;

    int[] dp = new int[n + 1];

    for (int i = 1; i <= m; i++) {
        int prev = 0; // 表示  $dp[i - 1][j - 1]$ 

        for (int j = 1; j <= n; j++) {
            int temp = dp[j]; // 保存旧  $dp[j]$ , 也就是  $dp[i - 1][j]$ 

            if (text1[i - 1] == text2[j - 1]) {
                dp[j] = prev + 1;
            } else {
                dp[j] = Math.Max(dp[j], dp[j - 1]);
            }

            prev = temp;
        }
    }

    return dp[n];
}
```

```
}
```

这里三个变量含义：

$dp[j]$ ：更新前是上一行，上方

$dp[j - 1]$ ：更新后是当前行，左方

prev：上一行左上角

面试回答模板

我会先定义状态：

$dp[...]$ 表示.....

然后分析选择：

当前状态可以由.....转移过来。

状态转移方程是：

$dp[...] = \dots$

初始化：

最小规模问题是.....

遍历顺序：

因为当前状态依赖.....，所以需要.....

返回结果：

题目要求的是.....，所以返回.....

比如面试讲 322 零钱兑换：

$dp[j]$ 表示凑成金额 j 所需的最少硬币数。

对于每个金额 j ，可以枚举每个 $coin$ ，从 $dp[j - coin]$ 转移过来。

所以 $dp[j] = \min(dp[j], dp[j - coin] + 1)$ 。

$dp[0] = 0$ ，其余初始化为 INF 。

最后返回 $dp[amount]$ ，如果仍然是 INF ，说明无法凑成，返回 -1 。